

Task 8:

Aim: To develop an Employee Management module using Spring Core demonstrating Inversion of Control and Dependency Injection using annotations, and managing beans using BeanFactory.

Algorithm:

- ☐ Start
- ☐ Create a Maven project
- ☐ Add Spring dependency in pom.xml
- ☐ Create package com.example
- ☐ Create classes:
 - Employee
 - EmployeeService
 - AppConfig
 - MainApp
- ☐ Annotate classes with @Component
- ☐ Use @Autowired for dependency injection
- ☐ Configure component scanning using @ComponentScan
- ☐ Initialize Spring container in MainApp
- ☐ Get bean object from container
- ☐ Call method to add and display employee data
- ☐ Display output on console
- ☐ Stop.

Program:

Pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```

<groupId>com.example</groupId>
<artifactId>employeeapp</artifactId>
<version>1.0-SNAPSHOT</version>
<dependencies>
    <!-- Spring Core -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.30</version>
    </dependency>
</dependencies>
</project>

```

Employee.java:

```

package com.example;

import org.springframework.stereotype.Component;

@Component
public class Employee {

    private int id;

    private String name;

    public void setId(int id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void display() {
        System.out.println("ID: " + id + ", Name: " + name);
    }
}

```

EmployeeService.java:

```
package com.example;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component

public class EmployeeService {

    @Autowired

    private Employee employee;

    public void addEmployee(int id, String name) {

        employee.setId(id);

        employee.setName(name);

        employee.display();

    }

}
```

MainApp.java:

```
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {

    public static void main(String[] args) {

        ApplicationContext context =

            new AnnotationConfigApplicationContext(AppConfig.class);

        EmployeeService service = context.getBean(EmployeeService.class);

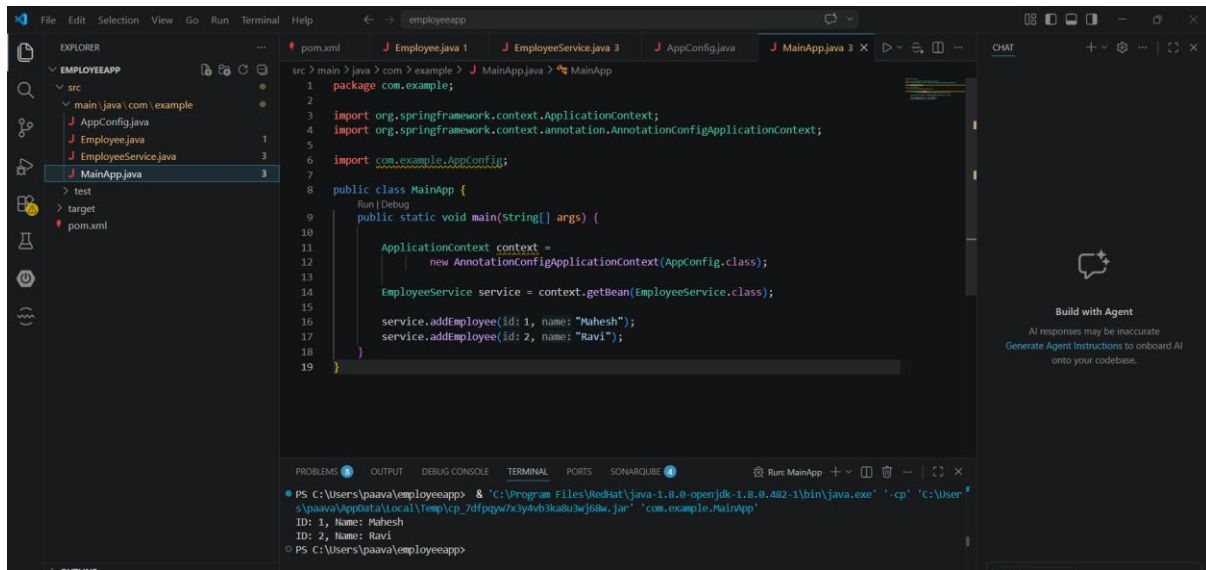
        service.addEmployee(1, "Mahesh");

        service.addEmployee(2, "Ravi");

    }

}
```

Output:



```
src > main > java > com > example > MainApp.java > MainApp
1 package com.example;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
5
6 import com.example.AppConfig;
7
8 public class MainApp {
9     public static void main(String[] args) {
10
11         ApplicationContext context =
12             new AnnotationConfigApplicationContext(AppConfig.class);
13
14         EmployeeService service = context.getBean(EmployeeService.class);
15
16         service.addEmployee(id: 1, name: "Mahesh");
17         service.addEmployee(id: 2, name: "Ravi");
18     }
19 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE

PS C:\Users\paava\employeeapp> & "C:\Program Files\RedHat\java-1.8.0-openjdk-1.8.0.482-1\bin\java.exe" "-cp" "C:\User
s\paava\AppData\Local\Temp\cp_7dfpqw7x3ydvb3ka8u3w\68w.jar" "com.example.MainApp"

ID: 1, Name: Mahesh
ID: 2, Name: Ravi

PS C:\Users\paava\employeeapp>

Result: Thus, to develop an Employee Management module using Spring Core demonstrating Inversion of Control and Dependency Injection using annotations, and managing beans using BeanFactory has been executed successfully.